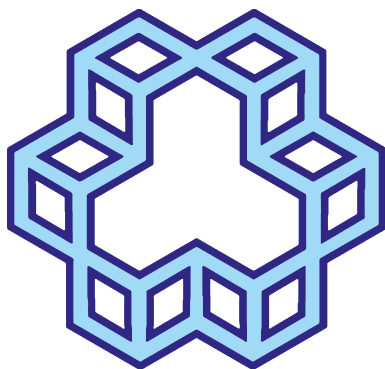




دانشگاه خواجه نصیرالدین طوسی

گزارش پروژه درس سیستم های عامل

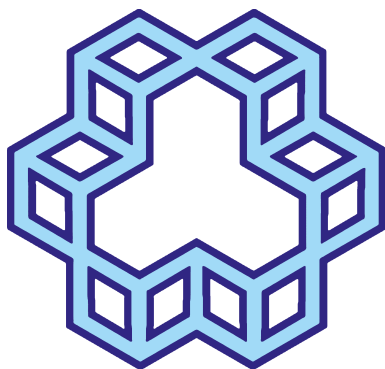
Safe Box

هدیه طهمورثی

استاد راهنما

دکتر یعقوبی

زمستان ۱۴۰۴



دانشگاه خواجه نصیرالدین طوسی

پروژه کارشناسی عنوان

Safe Box

نگارش

هدیه طهمورثی

استاد راهنما

دکتر یعقوبی

زمستان ۱۴۰۴



چکیده

امروزه کانتینرها زیرساخت اصلی رایانش ابری را تشکیل می‌دهند. سامانه SafeBox یک بستر اجرای کانتینر آموزشی است که با هدف بررسی عمیق مکانیزم‌های ایزولاسیون در هسته لینوکس طراحی شده است. این پروژه بدون استفاده از کتابخانه‌های آماده و صرفاً با تکیه بر زبان ++C و فراخوانی‌های سیستمی مستقیم پیاده‌سازی شده است. ویژگی‌های اصلی این سامانه شامل ایزولاسیون کامل پردازشی و فایلی با استفاده از فضاها نام، مدیریت دقیق منابع پردازنده و حافظه با استفاده از Cgroups v2 و ایمن‌سازی هسته با استفاده از فیلترهای Seccomp می‌باشد. نتایج آزمایش‌ها نشان می‌دهد که سامانه قادر است فرآیندهای کاربر را در محیطی امن و ایزوله اجرا نماید، هرچند محدودیت‌هایی در پایداری زیرسیستم شبکه در مقایسه با نمونه‌های صنعتی مشاهده شده است. این پروژه گامی موثر در جهت ابهام‌زدایی از فناوری‌های مجازی‌سازی سطح سیستم عامل است.

واژگان کلیدی کانتینر، لینوکس، ایزولاسیون، سیستم عامل، امنیت، SafeBox

فهرست مطالب

۱	مقدمه	فصل ۱:
۱	مقدمه	۱.۱
۱	بیان مسئله	۲.۱
۲	اهداف تحقیق	۳.۱
۲	ساختار گزارش	۴.۱
۳	مروری بر ادبیات موضوع و مبانی نظری	فصل ۲:
۳	مقدمه	۱.۲
۳	فضاهای نام لینوکس (The Walls)	۲.۲
۴	۱.۲.۲ فضای نام شناسه فرآیند (PID Namespace)	
۴	۲.۲.۲ فضای نام اتصال (Mount Namespace)	
۴	۳.۲.۲ فضای نام شبکه (Network Namespace)	
۵	۳.۲ گروه‌های کنترلی نسخه ۲ (The Ceiling)	
۵	۱.۳.۲ مکانیزم محدودسازی	
۵	۴.۲ ایزولاسیون فایل: chroot در برابر pivot_root	
۶	۵.۲ مدیریت شبکه در سطح هسته (Netlink)	
۶	۶.۲ فیلترینگ فراخوانی‌های سیستمی (Seccomp BPF)	
۷	معماری پیشنهادی و طراحی سیستم	فصل ۳:
۷	مقدمه	۱.۳

۲.۳	فلسفه طراحی: رویکرد «بدون جادو»	۷
۳.۳	معماری سطح بالا	۸
۴.۳	طراحی زیرسیستم‌ها	۸
۱.۴.۳	ایزولاسیون (Namespaces)	۸
۲.۴.۳	مدیریت منابع (Cgroups v2)	۹
۳.۴.۳	امنیت و فیلترینگ (Seccomp BPF)	۹
۵.۳	معماری رویت‌پذیری و ثبت وقایع	۱۰
۶.۳	طراحی زیرسیستم شبکه	۱۰
۱.۶.۳	انتخاب روش پیاده‌سازی: سوکت Netlink در برابر Wrapper	۱۰
۷.۳	جمع‌بندی	۱۱

فصل ۴: پیاده‌سازی و جزئیات فنی

۱.۴	مقدمه	۱۳
۲.۴	محیط توسعه و پیش‌نیازها	۱۳
۳.۴	پیاده‌سازی هسته مرکزی (Orchestrator)	۱۴
۱.۳.۴	پرچم‌های ایزولاسیون (Clone Flags)	۱۴
۲.۳.۴	سد همگام‌سازی (Synchronization Barrier)	۱۴
۴.۴	پیاده‌سازی زیرسیستم فایل (Filesystem)	۱۵
۵.۴	مدیریت منابع (CGroup Manager)	۱۵
۶.۴	زیرسیستم شبکه (Network Manager)	۱۶
۱.۶.۴	تکنیک پیاده‌سازی: فراخوانی‌های سیستمی (System Wrapper)	۱۶
۲.۶.۴	چالش واسط‌های زامبی (Zombie Interfaces)	۱۶
۷.۴	ماژول امنیت (Security)	۱۶
۸.۴	ارزیابی و تست (Evaluation)	۱۷
۱.۸.۴	تست ایزولاسیون (Isolation Test)	۱۷
۲.۸.۴	تست نفوذ (Breakout Test)	۱۷

۱۹	بحث و نتیجه‌گیری	فصل ۵:
۱۹	مقدمه	۱.۵
۱۹	تحلیل دستاوردها	۲.۵
۲۰	محدودیت‌ها و چالش‌ها	۳.۵
۲۰	ناپایداری زیرسیستم شبکه	۱.۳.۵
۲۰	کارایی سیستم فایل	۲.۳.۵
۲۱	عدم پیاده‌سازی فضای نام کاربر	۳.۳.۵
۲۱	پیشنهاد برای کارهای آتی	۴.۵
۲۲	جمع‌بندی	۵.۵
۲۳	کتاب‌نامه	
۲۵	راهنمای نصب و اجرا	پیوست آ:
۲۵	نصب پیش‌نیازها	۱.آ
۲۵	آماده‌سازی فایل سیستم	۲.آ
۲۶	کامپایل پروژه	۳.آ
۲۶	اجرای نمونه	۴.آ
۲۷	ساختار مخزن کد	پیوست ب:

فصل ۱

مقدمه

۱.۱ مقدمه

امروزه تکنولوژی‌های کانتینرسازی^۱ نظیر Docker و Kubernetes به استاندارد دفاکتو در صنعت نرم‌افزار تبدیل شده‌اند. این ابزارها با انتزاع پیچیدگی‌های سیستم‌عامل، امکان استقرار سریع و مقیاس‌پذیر نرم‌افزارها را فراهم می‌کنند. با این حال، برای بسیاری از دانشجویان و مهندسين سیستم، عملکرد داخلی این ابزارها همچنان مانند یک «جعبه سیاه»^۲ باقی مانده است.

پروژه SafeBox تلاشی است برای گشودن این جعبه سیاه. در این پروژه، به جای استفاده از ابزارهای آماده، یک سامانه کانتینر سبک از صفر با استفاده از زبان ++C و تعامل مستقیم با هسته لینوکس پیاده‌سازی شده است.

۲.۱ بیان مسئله

اجرای برنامه‌های غیرقابل اعتماد بر روی یک سرور مشترک، همواره با چالش‌های امنیتی و مدیریت منابع همراه است. اگر یک برنامه مخرب یا دارای خطا، کل حافظه رم سرور را مصرف کند یا فایل‌های سیستمی را حذف نماید، می‌تواند منجر به توقف سرویس‌دهی کل سرور شود. راهکارهای سنتی مانند ماشین‌های مجازی^۳

^۱Containerization ^۲Black Box ^۳Virtual Machines (VM)

با مجازی‌سازی سخت‌افزار، ایزولاسیون کاملی ارائه می‌دهند اما سربار منابع بسیار بالایی دارند. در مقابل، کانتینرها با استفاده از قابلیت‌های هسته لینوکس، ایزولاسیون را در سطح سیستم‌عامل و با سربار بسیار ناچیز فراهم می‌کنند. مسئله اصلی این گزارش، چگونگی پیاده‌سازی این ایزولاسیون بدون وابستگی به کتابخانه‌های سنگین خارجی است.

۳.۱ اهداف تحقیق

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک «زمان اجرا»^۱ کانتینر آموزشی-پژوهشی است که ویژگی‌های زیر را دارا باشد:

۱. ایزولاسیون فرآیند و فایل: هر کانتینر باید فضای PID و فایل سیستم اختصاصی خود را داشته باشد.
۲. کنترل منابع: امکان محدود کردن میزان مصرف پردازنده و حافظه برای هر کانتینر وجود داشته باشد.
۳. امنیت سطح هسته: جلوگیری از اجرای دستورات خطرناک سیستمی توسط کانتینر.
۴. استقلال پیاده‌سازی: عدم استفاده از کتابخانه‌هایی نظیر libcontainer یا runc برای درک عمیق مفاهیم.

۴.۱ ساختار گزارش

این گزارش در پنج فصل تدوین شده است. پس از این مقدمه، در فصل دوم به مبانی نظری کانتینرها در هسته لینوکس می‌پردازیم. فصل سوم معماری سیستم SafeBox را تشریح می‌کند. فصل چهارم به جزئیات دقیق پیاده‌سازی کدها و چالش‌های فنی اختصاص دارد و در نهایت فصل پنجم به بحث، نتیجه‌گیری و کارهای آتی می‌پردازد.

¹Runtime

فصل ۲

مروری بر ادبیات موضوع و مبانی نظری

۱.۲ مقدمه

تکنولوژی کانتینر^۱ برخلاف ماشین‌های مجازی^۲ که سخت‌افزار را شبیه‌سازی می‌کنند، از هسته سیستم عامل میزبان به صورت اشتراکی استفاده می‌کند. این تکنولوژی نوعی از مجازی‌سازی در سطح سیستم عامل^۳ است که بر پایه سه ستون اصلی در هسته Linux بنا شده است: فضاها^۴ نام^۴ برای ایزولاسیون، گروه‌های کنترلی^۵ برای مدیریت منابع، و مکانیزم‌های امنیتی نظیر Seccomp [۱]. در این فصل به بررسی دقیق این مکانیزم‌ها می‌پردازیم.

۲.۲ فضاها^۴ نام لینوکس (The Walls)

فضاها^۴ نام، منابع سراسری سیستم را پارتیشن‌بندی می‌کنند تا هر فرآیند نمای اختصاصی خود را از سیستم داشته باشد. این قابلیت اولین بار در سال ۲۰۰۲ با معرفی فضای نام Mount به هسته Linux اضافه شد [۲]. برای ایجاد یک کانتینر، معمولاً از فراخوانی سیستمی clone همراه با پرچم‌های^۶ خاصی نظیر CLONE_NEWPID یا CLONE_NEWNS استفاده می‌شود.

^۱Container ^۲Virtual Machines (VM) ^۳OS-level virtualization ^۴Namespaces ^۵Control Groups (Cgroups) ^۶Flags

۱.۲.۲ فضای نام شناسه فرآیند (PID Namespace)

در سیستم عامل Linux، شناسه‌های فرآیند^۱ به صورت سراسری یکتا هستند. اما با ایجاد یک فضای نام PID جدید، فرآیندها دارای دو شناسه می‌شوند: یک شناسه در فضای نام والد و یک شناسه در فضای نام جدید. فرآیند اولی که در فضای نام جدید ایجاد می‌شود، شناسه ۱ (PID 1) را دریافت می‌کند. این فرآیند نقش init را در کانتینر بازی می‌کند و وظیفه مدیریت سیگنال‌ها و جمع‌آوری فرآیندهای یتیم^۲ را بر عهده دارد. اگر این فرآیند متوقف شود، هسته تمام فرآیندهای دیگر درون آن فضای نام را از بین می‌برد.

۲.۲.۲ فضای نام اتصال (Mount Namespace)

این فضا مهم‌ترین رکن کانتینر است که به هر فرآیند اجازه می‌دهد نقطه اتصال^۳ های اختصاصی خود را داشته باشد. برخلاف دستور chroot که تنها ریشه را تغییر می‌دهد، این فضا کل درخت فایل سیستم را ایزوله می‌کند. یکی از مفاهیم کلیدی در اینجا، «انتشار اتصال»^۴ است. برای اینکه اتصال‌های درون کانتینر روی سیستم میزبان تاثیر نگذارند، باید نوع انتشار اتصال‌ها به حالت MS_PRIVATE تغییر یابد. این کار باعث می‌شود رویدادهای اتصال (Mount Events) درون کانتینر محبوس بمانند.

۳.۲.۲ فضای نام شبکه (Network Namespace)

این فضا یک پشته شبکه کاملاً ایزوله شامل کارت‌های شبکه^۵، جدول مسیریابی^۶ و قوانین فایروال iptables را ایجاد می‌کند. به طور پیش فرض، یک فضای نام جدید تنها دارای اینترفیس Loopback است و برای اتصال به خارج، نیاز به ایجاد زوج‌های اترنت مجازی^۷ و پل زنی^۸ به شبکه میزبان دارد.

^۱Process IDs (PIDs) ^۲Orphan Processes ^۳Mount Point ^۴Mount Propagation ^۵Network Interfaces
^۶Routing Table ^۷veth pairs ^۸Bridging

۳.۲ گروه‌های کنترلی نسخه ۲ (The Ceiling)

در حالی که فضاهای نام تعیین می‌کنند فرآیند «چه می‌بیند»، گروه‌های کنترلی تعیین می‌کنند فرآیند «چه مقدار می‌تواند مصرف کند». در نسخه اول Cgroups، هر منبع (مثل Memory یا CPU) سلسله‌مراتب جداگانه‌ای داشت که مدیریت آن را دشوار می‌کرد. اما در نسخه دوم (Cgroups v2) که در این پروژه استفاده شده است، تمامی کنترل‌کننده‌ها در یک «سلسله‌مراتب یکپارچه»^۱ مدیریت می‌شوند [۳].

۱.۳.۲ مکانیزم محدودسازی

مدیریت منابع در نسخه ۲ از طریق فایل‌های کنترلی در مسیر `/sys/fs/cgroup` انجام می‌شود. برای مثال:

- `memory.max`: حداکثر حافظه قابل استفاده بر حسب بایت. در صورت تجاوز از این مقدار، مکانیزم OOM Killer فرآیند را از بین می‌برد.
- `cpu.max`: سهمیه‌بندی زمان پردازنده. این فایل دو مقدار می‌گیرد: `quota` (زمان مجاز) و `period` (بازه زمانی). این مکانیزم تضمین می‌کند که کانتینر نمی‌تواند بیش از حد مجاز از سیکل‌های CPU استفاده کند.

۴.۲ ایزولاسیون فایل: chroot در برابر pivot_root

یکی از اشتباهات رایج، یکسان دانستن chroot با کانتینر است. دستور chroot تنها دایرکتوری ریشه را تغییر می‌دهد اما سیستم‌فایل قبلی را جدا نمی‌کند. اگر فرآیندی دسترسی به یک توصیف‌گر فایل^۲ باز از خارج محیط داشته باشد، می‌تواند از زندان فرار کند (Jailbreak).

روش استاندارد و امن، استفاده از pivot_root است. این فراخوانی سیستمی، ریشه اتصال فعلی را به یک دایرکتوری داخلی منتقل کرده و ریشه جدید را جایگزین آن می‌کند. سپس با استفاده از دستور `umount2` با پرچم

^۱Unified Hierarchy^۲File Descriptor

MNT_DETACH، ریشه قدیمی به طور کامل از فضای نام جدا می‌شود، به طوری که فرآیند هیچ راهی برای بازگشت به فایل سیستم میزبان ندارد [۴].

۵.۲ مدیریت شبکه در سطح هسته (Netlink)

استاندارد صنعتی برای پیکربندی شبکه در Linux، استفاده از پروتکل Netlink است. برخلاف روش‌های قدیمی مبتنی بر ioctl، پروتکل Netlink یک سرویس پیام‌رسان دوطرفه بین فضای کاربر^۱ و فضای هسته^۲ است. این پروتکل با استفاده از سوکت‌های AF_NETLINK امکان ساخت کارت‌های شبکه مجازی، تغییر آدرس‌های IP و گوش دادن به رویدادهای شبکه را به صورت آسنکرون فراهم می‌کند. ابزارهایی مانند Docker و بسته iproute2 (دستور ip) از این مکانیزم برای مدیریت پویا و سریع شبکه استفاده می‌کنند [۵].

۶.۲ فیلترینگ فراخوانی‌های سیستمی (Seccomp BPF)

هسته Linux بیش از ۴۰۰ فراخوانی سیستمی دارد که بسیاری از آن‌ها برای یک کانتینر ضروری نیستند و حتی می‌توانند خطرناک باشند (مانند reboot، swapon یا kexec_load). مکانیزم Seccomp به فرآیند اجازه می‌دهد تا با استفاده از فیلترهای BPF، یک سیاست امنیتی تعریف کند. این فیلترها که در واقع بایت‌کدهای ماشین مجازی BPF هستند، روی شماره فراخوانی سیستمی تصمیم‌گیری می‌کنند. سیاست‌ها معمولاً به دو دسته تقسیم می‌شوند:

۱. **Whitelist**: مسدود کردن همه چیز (SCMP_ACT_KILL) و مجاز کردن موارد خاص. (بسیار امن اما دشوار در پیاده‌سازی)

۲. **Blacklist**: مجاز کردن همه چیز (SCMP_ACT_ALLOW) و مسدود کردن موارد خطرناک. (پیاده‌سازی آسان‌تر اما امنیت کمتر)

پروژه SafeBox برای تعادل بین امنیت و کارایی، از رویکرد دوم استفاده کرده است [۶].

¹User Space ²Kernel Space

فصل ۳

معماری پیشنهادی و طراحی سیستم

۱.۳ مقدمه

در این فصل به تشریح معماری سامانه SafeBox و تصمیمات فنی اتخاذ شده در طراحی آن می‌پردازیم. هدف اصلی در طراحی این سامانه، ایجاد یک بستر ایزولاسیون^۱ سبک و امن برای اجرای فرآیندهای غیرقابل اعتماد بوده است. لایه‌بندی شده و پیچیده بنا شده‌اند، معماری SafeBox به صورت یکپارچه و با تعامل مستقیم با هسته لینوکس طراحی شده است.

۲.۳ فلسفه طراحی: رویکرد «بدون جادو»

یکی از اصول بنیادین در توسعه این پروژه، فلسفه «بدون جادو»^۲ بوده است. در اکوسیستم کانتینرهای مدرن، ابزارهایی نظیر runc و containerd پیچیدگی‌های تعامل با هسته سیستم‌عامل را انتزاع می‌کند. در SafeBox، تصمیم بر آن شد که از هیچ کتابخانه شخص ثالثی برای مدیریت کانتینر استفاده نشود. تمامی تعاملات با هسته لینوکس - از ایجاد فضای نام تا اعمال محدودیت‌های سخت‌افزاری - مستقیماً از طریق فراخوانی‌های سیستمی^۳ و زبان C++17 پیاده‌سازی شده‌اند. این رویکرد اگرچه پیچیدگی پیاده‌سازی را افزایش

^۱Isolation ^۲No Magic ^۳System Calls

می‌دهد، اما کنترل دقیقی بر چرخه حیات فرایندها فراهم می‌کند.

۳.۳ معماری سطح بالا

معماری سیستم بر پایه الگوی «مدیر-کارگر»^۱ طراحی شده است. در این مدل، یک فرآیند والد (مدیر) وظیفه آماده‌سازی محیط و نظارت را بر عهده دارد و فرآیند فرزند (کارگر) درون محیط ایزوله اجرا می‌شود. مولفه مرکزی سیستم، کلاس Container است که به عنوان «ارکستراتور»^۲ عمل می‌کند. این کلاس وظیفه هماهنگی بین زیرسیستم‌های مختلف (شبکه، فایل، امنیت) را بر عهده دارد. چرخه حیات یک کانتینر در این معماری شامل مراحل زیر است:

۱. پیکربندی: دریافت آرگومان‌های ورودی و ساخت آبجکت تنظیمات.
۲. انشعاب فرآیند: استفاده از فراخوانی clone برای ایجاد فرزند در فضاهای نام جدید.
۳. سد همگام‌سازی: توقف فرآیند فرزند تا زمان اعمال محدودیت‌ها توسط والد.
۴. اعمال محدودیت: تنظیم Cgroups و شبکه توسط والد.
۵. راه‌اندازی محیط: تغییر ریشه فایل (pivot_root) و اعمال فیلترهای امنیتی توسط فرزند.
۶. اجرا: جایگزینی فرآیند فرزند با دستور کاربر توسط execvp.

۴.۳ طراحی زیرسیستم‌ها

۱.۴.۳ ایزولاسیون (Namespaces)

برای جداسازی کامل محیط اجرا، از قابلیت فضاهای نام^۳ در هسته لینوکس استفاده شده است. طراحی ما چهار فضای نام اصلی را هدف قرار داده است:

¹Manager-Worker Pattern

²Orchestrator

³Linux Namespaces

- **PID Namespace:** برای ایزوله کردن شماره فرایندها. فرآیند درون کانتینر خود را با شناسه ۱ (PID 1) می‌بیند که برای مدیریت سیگنال‌ها و چرخه حیات ضروری است.
- **Mount Namespace:** برای ایجاد یک نمای اختصاصی از فایل سیستم. هرگونه عملیات mount درون کانتینر بر سیستم میزبان تاثیر نمی‌گذارد.
- **UTS Namespace:** برای اختصاص نام میزبان (Hostname) مستقل.
- **Network Namespace:** برای ایزوله کردن کامل پشته شبکه (اینترفیس‌ها، جداول مسیریابی).

۲.۴.۳ مدیریت منابع (Cgroups v2)

برای جلوگیری از انحصار منابع توسط کانتینر، از نسخه دوم گروه‌های کنترلی^۱ استفاده شده است. معماری v2 به دلیل ساختار سلسله‌مراتبی یکپارچه انتخاب گردید که مدیریت همزمان پردازنده و حافظه را تسهیل می‌کند. طراحی ماژول مدیریت منابع به گونه‌ای است که پیش از شروع به کار فرآیند فرزند، محدودیت‌ها اعمال می‌شوند. این امر از بروز شرایطی که فرآیند فرزند قبل از محدود شدن، کل حافظه سیستم را مصرف کند (Race Condition)، جلوگیری می‌کند.

۳.۴.۳ امنیت و فیلترینگ (Seccomp BPF)

امنیت در سطح هسته توسط مکانیزم Seccomp BPF تامین می‌شود. در طراحی این بخش، دو رویکرد «لیست سفید»^۲ و «لیست سیاه»^۳ مورد بررسی قرار گرفت. اگرچه رویکرد لیست سفید (مسدود کردن همه چیز و فقط مجاز کردن موارد خاص) امن‌تر است، اما به دلیل نیاز به شناسایی دقیق صدها فراخوانی سیستمی مورد نیاز برای اجرای یک پوسته ساده، رویکرد لیست سیاه انتخاب شد. در این طراحی، به صورت پیش فرض همه فراخوانی‌ها مجاز هستند، اما موارد خطرناک نظیر `swapon`، `reboot` و مدیریت ماژول‌های هسته صراحتاً مسدود می‌شوند.

^۱Control Groups v2 ^۲Whitelist ^۳Blacklist

۵.۳ معماری رویت‌پذیری و ثبت وقایع

یکی از چالش‌های اصلی در طراحی سیستم‌های ایزوله، مساله «شکست خاموش»^۱ است. زمانی که فرزند فرزند ریشه فایل سیستم خود را تغییر می‌دهد (pivot_root)، دسترسی به فایل‌های لاگ موجود در سیستم میزبان را از دست می‌دهد. برای حل این مشکل، معماری ثبت وقایع (Logging) به صورت ترکیبی طراحی شده است:

۱. خروجی استاندارد خطا (stderr): از آنجا که توصیف‌گرهای فایل^۲ از فرزند والد به ارث می‌رسند، پیام‌های خطا مستقیماً به کنسول هدایت می‌شوند تا حتی در صورت از دست رفتن فایل سیستم، مدیر سیستم از خطا مطلع شود.

۲. فایل لاگ: تلاش برای ثبت وقایع در فایل دیسک به عنوان روش تکمیلی.

۶.۳ طراحی زیرسیستم شبکه

طراحی شبکه در SafeBox بر اساس مدل استاندارد Bridge انجام شده است. در این مدل، کانتینرها از طریق یک زوج اینترفیس مجازی (veth pair) به یک پل مجازی (Linux Bridge) روی سیستم میزبان متصل می‌شوند.

۱.۶.۳ انتخاب روش پیاده‌سازی: سوکت Netlink در برابر Wrapper

در پیاده‌سازی این ماژول، یک تصمیم معماری مهم اتخاذ شد. روش استاندارد و حرفه‌ای برای تعامل با پشته شبکه لینوکس، استفاده از سوکت‌های Netlink است. با این حال، به دلیل پیچیدگی بالای پروتکل Netlink و محدودیت‌های زمانی پروژه، تصمیم گرفته شد از رویکرد «پوسته واسطه»^۳ استفاده شود. در این رویکرد، سامانه دستورات استاندارد iproute2 (مانند ip link add) را از طریق فراخوانی system() اجرا می‌کند. اگرچه این روش برای یک محیط عملیاتی ایده‌آل نیست (سربار ایجاد فرآیند جدید)، اما برای اثبات مفاهیم معماری شبکه و مسیریابی در یک پروژه دانشگاهی، راه‌حلی کارآمد و سریع محسوب می‌شود.

¹Silent Failure²File Descriptors³Shell Wrapper / System Calls

۷.۳ جمع‌بندی

معماری SafeBox تلاشی برای ایجاد تعادل بین سادگی آموزشی و کارایی فنی است. با پرهیز از کتابخانه‌های آماده و تعامل مستقیم با هسته، این معماری امکان درک عمیق مفاهیم سیستم‌عامل را فراهم می‌کند. استفاده از مکانیزم‌های مدرن مانند Cgroups v2 و جداسازی کامل فضاها نام، این پروژه را به یک شبیه‌سازی کوچک اما واقعی از کانتینرهای صنعتی تبدیل کرده است.

فصل ۴

پیاده‌سازی و جزئیات فنی

۱.۴ مقدمه

در این فصل به تشریح جزئیات پیاده‌سازی سامانه SafeBox می‌پردازیم. برخلاف فصل قبل که بر معماری تمرکز داشت، در اینجا کدها، توابع سیستمی و چالش‌های فنی واقعی مورد بررسی قرار می‌گیرند. این سامانه با استفاده از زبان C++17 توسعه یافته و برای مدیریت ساختار بیلد از CMake استفاده می‌کند.

۲.۴ محیط توسعه و پیش‌نیازها

برای پیاده‌سازی این پروژه از ابزارهای استاندارد لینوکس استفاده شده است. انتخاب توزیع Alpine Linux به عنوان سیستم‌فایل ریشه (RootFS) به دلیل حجم بسیار کم (کمتر از ۵ مگابایت) و عدم وابستگی به systemd بوده است که فرآیند بوت شدن کانتینر را سریع و ساده می‌کند. اسکریپت setup_fs.sh وظیفه دانلود و استخراج خودکار این فایل سیستم را بر عهده دارد:

```
wget -q0 alpine.tar.gz https://dl-cdn.alpinelinux.org/.../alpine...tar.gz
mkdir -p ./rootfs
tar -xzf alpine.tar.gz -C ./rootfs
```

۳.۴ پیاده‌سازی هسته مرکزی (Orchestrator)

کلاس Container در فایل src/core/Container.cpp قلب تپنده سیستم است. مهم‌ترین وظیفه این کلاس، ایجاد فرآیند فرزند با استفاده از فراخوانی سیستمی clone است.

۱.۳.۴ پرچم‌های ایزولاسیون (Clone Flags)

ما از پرچم‌های زیر برای ایجاد فضاها با نام جدید استفاده کرده‌ایم:

```
clone(child_func, stack_top,
      CLONE_NEWUTS | CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWNET | SIGCHLD,
      &args);
```

این پرچم‌ها تضمین می‌کنند که فرزند در یک محیط ایزوله از نظر نام میزبان، شناسه فرآیند، فایل سیستم و شبکه متولد می‌شود.

۲.۳.۴ سد همگام‌سازی (Synchronization Barrier)

یکی از چالش‌های پیاده‌سازی، جلوگیری از اجرای کد کاربر قبل از اعمال محدودیت‌های منابع بود. اگر فرزند بلافاصله پس از clone شروع به اجرا کند، ممکن است قبل از اینکه والد فرصت تنظیم Cgroups را داشته باشد، تمام حافظه سیستم را مصرف کند (شرایط مسابقه). برای حل این مشکل، از یک Pipe یونیکس استفاده کردیم:

۱. والد یک پایپ ایجاد می‌کند.

۲. فرزند پس از clone، روی خواندن از پایپ مسدود (Block) می‌شود.

۳. والد تنظیمات شبکه و Cgroups را اعمال می‌کند.

۴. والد یک بایت در پایپ می‌نویسد تا فرزند آزاد شده و به اجرا ادامه دهد.

۴.۴ پیاده‌سازی زیرسیستم فایل (Filesystem)

برای ایجاد زندان فایل (Chroot Jail) امن، به جای دستور قدیمی chroot از pivot_root استفاده شده است. پیاده‌سازی در src/core/Filesystem.cpp شامل مراحل زیر است:

۱. **خصوصی سازی Mount:** با استفاده از MS_PRIVATE اطمینان حاصل می‌شود که تغییرات مانت به سیستم میزبان نشت نمی‌کند.
۲. **آماده سازی Bind Mount:** دایرکتوری rootfs به خودش bind می‌شود (الزام pivot_root).
۳. **جابجایی ریشه:**

```
syscall(SYS_pivot_root, new_root, put_old);
```

۴. **حذف ریشه قدیم:** دایرکتوری old_root که حاوی فایل سیستم میزبان بود، unmount می‌شود تا دسترسی به میزبان کاملاً قطع گردد.
۵. **مانت کردن proc:** برای عملکرد صحیح دستوراتی مثل ps، فایل سیستم procfs مجدداً در /proc مانت می‌شود.

۵.۴ مدیریت منابع (CGroup Manager)

ماژول CGroupManager با فایل سیستم مجازی cgroup v2 تعامل دارد. برای محدود کردن پردازنده، از مدل سهمیه‌بندی (Quota/Period) استفاده می‌شود. اگر کاربر محدودیت ۵۰ درصد را درخواست کند، مقادیر زیر محاسبه و در فایل cpu.max نوشته می‌شود:

$$Period = 100000\mu s, \quad Quota = 50000\mu s \quad (1.4)$$

این کار تضمین می‌کند فرآیند کانتینر در هر بازه زمانی ۱۰۰ میلی‌ثانیه‌ای، تنها ۵۰ میلی‌ثانیه از پردازنده استفاده کند.

۶.۴ زیرسیستم شبکه (Network Manager)

در بخش طراحی اشاره شد که شبکه از طریق پل زنی (Bridge) انجام می‌شود.

۱.۶.۴ تکنیک پیاده‌سازی: فراخوانی‌های سیستمی (System Wrapper)

اگرچه استاندارد صنعتی استفاده از سوکت‌های Netlink است، اما در این پروژه برای تسریع در نمونه‌سازی اولیه از فراخوانی دستورات ip توسط تابع system() استفاده شد. کد زیر در src/core/NetworkManager.cpp وظیفه ساخت پل شبکه را بر عهده دارد:

```
run_cmd("ip link add name sb-br0 type bridge");
run_cmd("ip addr add 10.0.0.1/24 dev sb-br0");
run_cmd("ip link set sb-br0 up");
```

۲.۶.۴ چالش واسط‌های زامبی (Zombie Interfaces)

یکی از محدودیت‌های مهم این روش، مدیریت وضعیت (State Management) است. اگر برنامه SafeBox به دلیل خطا (Crash) بسته شود، دستورات پاکسازی اجرا نمی‌شوند و اینترفیس sb-br0 روی سیستم باقی می‌ماند. این پدیده باعث می‌شود در اجرای بعدی، برنامه با خطای «تکراری بودن نام» (EEXIST) مواجه شود. در سیستم‌های صنعتی، یک دیمون (Daemon) جداگانه وظیفه پایش و پاکسازی این منابع یتیم را بر عهده دارد.

۷.۴ ماژول امنیت (Security)

برای کاهش سطح حمله، از libseccomp برای بارگذاری فیلترهای BPF در هسته استفاده شد. سیاست امنیتی به صورت «لیست سیاه» پیاده‌سازی شده است که در آن فراخوانی‌های زیر صراحتاً مسدود شده‌اند:

- `reboot`: جلوگیری از ری‌استارت کردن سرور میزبان توسط کانتینر.
- `swapon/swapoff`: جلوگیری از تغییر تنظیمات حافظه مجازی.
- `init_module`: جلوگیری از بارگذاری ماژول‌های هسته مخرب.

در صورت فراخوانی این دستورات، فرآیند بلافاصله توسط هسته با سیگنال SIGSYS کشته می‌شود.

۸.۴ ارزیابی و تست (Evaluation)

برای اطمینان از صحت عملکرد، دو سناریوی تست اصلی اجرا شد.

۱.۸.۴ تست ایزولاسیون (Isolation Test)

با اجرای یک پوسته `/bin/sh` درون کانتینر و دستور `ps aux`، خروجی زیر مشاهده شد:

PID	USER	TIME	COMMAND
1	root	0:00	/bin/sh
2	root	0:00	ps aux

این خروجی نشان می‌دهد که فضای نام PID به درستی عمل کرده و پوسته تنها خودش را می‌بیند (به عنوان PID ۱). همچنین دستور `ls` تنها محتویات فایل سیستم Alpine را نمایش داد و اثری از فایل‌های میزبان نبود.

۲.۸.۴ تست نفوذ (Breakout Test)

برای ارزیابی ماژول امنیتی، یک برنامه اختصاصی به نام `breakout.cpp` نوشته شد که تلاش می‌کند سیستم میزبان را ری‌بوت کند:

```
// tests/breakout.cpp
syscall(SYS_reboot, ...);
```

هنگام اجرای این برنامه درون کانتینر، سیستم عامل بلافاصله فرآیند را متوقف کرد و پیام زیر در لاگ‌های dmesg سیستم میزبان ثبت شد که نشان‌دهنده عملکرد صحیح فیلتر Seccomp است.

فصل ۵

بحث و نتیجه‌گیری

۱.۵ مقدمه

در این پایان‌نامه، مراحل طراحی و پیاده‌سازی سامانه SafeBox، یک بستر اجرای کانتینر سبک مبتنی بر سیستم‌عامل Linux، تشریح گردید. هدف اصلی این پژوهش، ابهام‌زدایی از فناوری‌های مجازی‌سازی سطح سیستم‌عامل و ارائه یک پیاده‌سازی آموزشی بدون وابستگی به کتابخانه‌های شخص ثالث بود. سامانه نهایی موفق شد با استفاده از زبان C++ و تعامل مستقیم با فراخوانی‌های سیستمی، محیطی ایزوله را برای اجرای فرآیندها فراهم کند که دارای سیستم‌فایل اختصاصی، محدودیت منابع سخت‌افزاری و فیلترینگ امنیتی است.

۲.۵ تحلیل دستاوردها

پروژه SafeBox نشان داد که با کمتر از ۱۰۰۰ خط کد، می‌توان یک موتور کانتینر کاربردی ایجاد کرد. دستاوردهای کلیدی عبارتند از:

۱. ایزولاسیون کامل: استفاده صحیح از پرچم‌های clone (شامل PID، UTS، Mount و Net) باعث شد فرآیند فرزند هیچ دیدی نسبت به فرآیندهای میزبان نداشته باشد.
۲. مدیریت منابع مدرن: پیاده‌سازی موفق Cgroups v2 امکان اعمال محدودیت‌های دقیق بر مصرف

حافظه و پردازنده را فراهم کرد.

۳. امنیت پیشگیرانه: استفاده از Seccomp BPF به عنوان لایه دفاعی آخر، مانع از اجرای دستورات مخرب مانند reboot توسط کانتینر شد.

۳.۵ محدودیت‌ها و چالش‌ها

علیرغم موفقیت در اهداف اصلی، این سامانه به عنوان یک پروژه دانشگاهی دارای محدودیت‌هایی است که آن را از نمونه‌های صنعتی متمایز می‌کند.

۱.۳.۵ ناپایداری زیرسیستم شبکه

همان‌طور که در فصل چهارم اشاره شد، برای مدیریت شبکه از روش «پوسته واسط»^۱ استفاده شده است. اگرچه این روش برای نمونه‌سازی سریع مناسب است، اما دو مشکل اساسی دارد:

- **کارایی:** اجرای مکرر دستور `system()` و ایجاد فرآیندهای جدید برای هر دستور `ip` سربار زمانی ایجاد می‌کند.
- **مدیریت وضعیت:** در صورت بسته شدن ناگهانی برنامه (Crash)، اینترفیس‌های شبکه مجازی (veth pairs) و پل‌های شبکه (Bridge) روی سیستم میزبان باقی می‌مانند (اینترفیس‌های زامبی). از آنجا که SafeBox فاقد یک دیمون^۲ نظارتی است، این منابع باید به صورت دستی پاکسازی شوند.

۲.۳.۵ کارایی سیستم فایل

سامانه فعلی از کپی‌برداری ساده دایرکتوری ریشه (Directory Binding) استفاده می‌کند. این بدان معناست که هر کانتینر فضای دیسک جداگانه‌ای اشغال می‌کند. برخلاف Docker که از فایل سیستم‌های لایه‌ای مانند

^۱Shell Wrapper ^۲Daemon

OverlayFS برای اشتراک‌گذاری لایه‌های پایه بین کانتینرها استفاده می‌کند، رویکرد فعلی از نظر مصرف دیسک بهینه نیست.

۳.۳.۵ عدم پیاده‌سازی فضای نام کاربر

در نسخه فعلی، از فضای نام کاربر^۱ استفاده نشده است. این بدان معناست که فرآیند کانتینر، اگرچه ایزوله است، اما همچنان با شناسه کاربری ریشه (UID 0) اجرا می‌شود. اگر یک نفوذگر بتواند از حصار کانتینر فرار کند، دسترسی ریشه به سیستم میزبان خواهد داشت.

۴.۵ پیشنهاد برای کارهای آتی

برای ارتقای سامانه SafeBox به یک ابزار عملیاتی‌تر، موارد زیر پیشنهاد می‌گردد:

۱. **مهاجرت به Netlink:** جایگزینی فراخوانی‌های system() با سوکت‌های Netlink برای مدیریت شبکه. این کار امکان مدیریت تراکنش‌دار منابع شبکه و دریافت رویدادهای تغییر وضعیت را فراهم می‌کند.

۲. **پیاده‌سازی Daemon مرکزی:** طراحی یک فرآیند دیمون که مستقل از چرخه حیات کانتینرها اجرا شود و وظیفه پاکسازی منابع یتیم (مانند اینترفیس‌های شبکه و گروه‌های کنترلی) را بر عهده داشته باشد.

۳. **پشتیبانی از OverlayFS:** پیاده‌سازی منطق اتصال لایه‌ای برای کاهش مصرف دیسک و امکان استفاده از ایمیج‌های استاندارد OCI.

۴. **کانتینرهای بدون ریشه: (Rootless)** پیاده‌سازی فضای نام کاربر (CLONE_NEWUSER) برای نگاشت شناسه کاربری ریشه درون کانتینر به یک کاربر عادی در سیستم میزبان، که امنیت سامانه را به شدت افزایش می‌دهد.

¹User Namespace

۵.۵ جمع‌بندی

پروژه SafeBox تلاشی موفق برای درک عمیق مفاهیم سیستم‌عامل بود. این پژوهش نشان داد که «کانتینر» یک شیء فیزیکی واحد نیست، بلکه ترکیبی هوشمندانه از مکانیزم‌های مختلف هسته Linux است که در کنار هم، توهم یک سیستم مستقل را ایجاد می‌کنند. کد منبع این پروژه می‌تواند به عنوان یک مرجع آموزشی برای دانشجویان درس سیستم‌عامل مورد استفاده قرار گیرد.

کتاب نامه

- [1] Rice, Liz. *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, 2020.
- [2] Kerrisk, Michael. Linux programmer's manual: Namespaces(7). <https://man7.org/linux/man-pages/man7/namespaces.7.html>, 2024. Accessed: 2026-02-02.
- [3] Heo, Tejun. Control group v2 documentation. <https://www.kernel.org/doc/Documentation/cgroup-v2.txt>, 2015. Linux Kernel Organization.
- [4] Organization, Linux Kernel. pivot_root(2) - linux manual page. https://man7.org/linux/man-pages/man2/pivot_root.2.html.
- [5] Salim, J., Khosravi, H., and Kleen, A. *RFC 3549: Linux Netlink as an IP Services Protocol*. IETF, 2003.
- [6] Corbet, Jonathan. Seccomp bpf: Sandboxing for the masses. *LWN.net*, 2012.

پیوست آ

راهنمای نصب و اجرا

برای کامپایل و اجرای پروژه SafeBox به یک توزیع لینوکس مدرن (با هسته نسخه ۴.۵ به بالا) و کامپایلر G++ با پشتیبانی از استاندارد C++17 نیاز دارید.

۱. آ نصب پیش نیازها

ابتدا پکیج‌های لازم را نصب کنید:

```
sudo apt-get update
```

```
sudo apt-get install -y build-essential cmake libseccomp-dev wget tar
```

۲. آ آماده‌سازی فایل سیستم

اسکرپت آماده‌سازی را اجرا کنید تا ایمج Alpine Linux دانلود شود:

```
chmod +x setup_fs.sh
```

```
./setup_fs.sh
```

آ.۳ کامپایل پروژه

```
mkdir build && cd build
```

```
cmake ..
```

```
make
```

آ.۴ اجرای نمونه

به دلیل استفاده از قابلیت‌های هسته، برنامه باید با دسترسی root اجرا شود:

```
sudo ./safebox --cmd /bin/sh --mem 50 --cpu 20
```

پیوست ب

ساختار مخزن کد

ساختار فایل‌ها و دایرکتوری‌های پروژه به شرح زیر است:

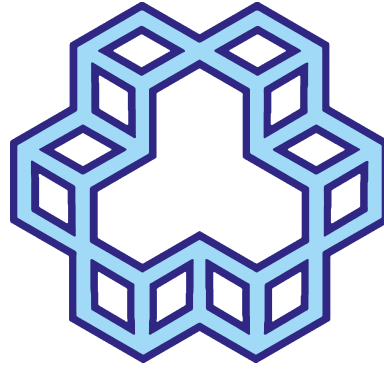
```
SafeBox/
  CMakeLists.txt      # Build configuration
  setup_fs.sh         # RootFS downloader script
  include/
    core/
      Container.h      # Main orchestration logic
      CGroupManager.h  # Resource limits interface
      NetworkManager.h # Network bridge interface
      Security.h        # Seccomp BPF interface
      Filesystem.h      # Mount & pivot_root logic
      Monitor.h         # Stats collection
  src/
    main.cpp           # CLI Entry point
    core/
      Container.cpp
```

```
CGroupManager.cpp
NetworkManager.cpp
Security.cpp
Filesystem.cpp
Monitor.cpp
tests/
    breakout.cpp          # Security validation test
rootfs/                  # Guest OS files (created by setup script)
```


Abstract

SafeBox is a custom-developed container runtime for Linux systems, engineered to demonstrate the fundamental mechanisms of process isolation, resource management, and kernel-level security. Unlike commercial engines that abstract away underlying implementations, SafeBox is built from scratch in C++ to expose the raw interaction with Linux Namespaces, Control Groups (v2), and Seccomp BPF. The system implements a process orchestrator capable of launching isolated environments with restricted file system access and resource limits. While the project successfully demonstrates the core pillars of containerization— isolation and resource control—it also highlights the engineering complexities of network virtualization. This thesis details the architectural decisions and the raw kernel interfaces utilized to build a functional container engine.

Keywords



K. N. Toosi University of Technology
- Operating System Course

Project Report on

SafeBox

By:

Hedie Tahmouresi

Supervisor:

Dr. Yaghoobi

Winter 2026